Building Real-Time Voice Agents

A practical guide to designing voice pipelines, managing latency, handling interruptions, and delivering natural spoken experiences on top of existing agent logic.

VOICE AI | PIPELINE DESIGN | REAL-TIME SYSTEMS

What Makes a Voice Agent Different?

It's not just text with audio wrappers

If the same agent logic can answer both text and voice, what actually changes when you move to voice? More than you might expect.

Continuous Audio, Not Discrete Messages

A voice agent must handle **continuous audio**, not just discrete text messages. Users expect it to listen, think, and speak in near real time — there is no "send" button.

Timing Is Part of the Product

Turn-taking becomes a core design concern. Users notice delays, overlaps, and mis-timed interruptions far more acutely than they do in text-based interactions.

⚠ **You cannot just "wrap it with STT/TTS."** You need to design how the agent behaves over time — how early it responds, when it asks for confirmation, and how it handles interruptions.

The Core Stages of a Modern Voice Pipeline

When you look under the hood, a typical voice agent runs through three tightly coupled stages — each feeding into the next in a continuous stream.

STT

Speech-to-Text: streaming transcription that emits partial and final text as the user speaks, before they finish.

Agent / LLM

Interprets the transcript, calls tools or other agents, and **streams a response** token by token.

TTS

Text-to-Speech: converts tokens into speech, often while the agent is still generating the rest of the reply.

STT feeds partial transcripts to the agent, which can start planning and responding **before the user finishes speaking**. TTS then streams spoken output as soon as there is enough text to say — all three stages overlap in time.

The Real-Time Streaming Pipeline: A Deep Dive

Natural voice agents stay responsive because every stage works continuously and overlaps with the others.

STT Stage

Emits partial transcripts every ~100–300ms as the user speaks.

- Final transcript sent on end-of-utterance
- Partial transcripts enable early agent planning
- Final transcript is used for action
- Key concern: shifting partials as the model refines text

Agent / LLM Stage

Receives partial transcripts as a stream and starts reasoning immediately.

- Begins generating tokens before the user finishes
- Streams tokens out immediately
- Does not wait for the full response to be ready
- Detects sentence boundaries to chunk output for TTS

TTS Stage

Receives token chunks, not full text, and starts speaking right away.

- Synthesizes audio per sentence or chunk
- Begins playback before the full response is generated
- Buffers a small amount of audio to smooth playback
- Keeps the conversation feeling fluid and immediate

ⓘ The key insight: all three stages overlap in time. STT, LLM, and TTS are all active simultaneously during a turn — the pipeline is a stream of streams, not a sequence of steps.

Alternative Voice Pipeline Architectures

While the three-stage STT→LLM→TTS pipeline is standard, new architectural patterns are emerging to push the boundaries of real-time voice interaction, each with its own trade-offs.

Feature	Classic (STT→LLM→TTS)	Native Speech-to-Speech	Hybrid Approach
Latency	Additive across stages; noticeable delays can occur.	Lowest; audio in, audio out directly from model for fastest response.	Medium; fast STT, then direct audio output, bypassing separate TTS.
Control/Modularity	High; each component (STT, LLM, TTS) is swappable and configurable.	Low; tightly integrated model, less control over individual stages.	Medium; STT is swappable, but the multimodal model is integrated.
Naturalness	Good, but can sometimes sound robotic or lack emotional nuance.	Excellent; model-generated prosody and emotion for highly natural speech.	Very Good; multimodal model generates audio directly, improving on TTS.
Cost	Variable, depends on individual service providers for each stage.	High; premium, advanced multimodal models are typically more expensive.	Medium; involves a fast STT and a capable multimodal LLM.
Maturity	High; well-established and widely implemented in production.	Emerging; limited model availability and rapidly evolving capabilities.	Emerging; relies on advanced multimodal models and fast STT technology.

When to Choose: The **Classic** pipeline offers flexibility for most production needs. **Native Speech-to-Speech** is ideal for highly natural, low-latency, immersive experiences where model choice isn't a constraint. The **Hybrid** approach balances naturalness and speed with some modularity, leveraging advanced multimodal LLMs.

Latency Budgeting: Why Every Millisecond Counts

What happens without a latency budget?

The experience quickly feels laggy. Users finish speaking, wait, and wonder if anything is happening — because STT, agent, and TTS delays **add up silently**. Even a great answer feels bad if the first audible response comes after a long pause.

STT

A few hundred ms for streaming partial transcripts

Agent

A few hundred ms for reasoning and tool calls

TTS

First audio within another few hundred ms

How to think about it as a budget

Roughly allocate time across stages and optimise where you exceed the budget. The key principle: **start streaming as early as possible** at every stage.

- Stream partial transcripts from STT immediately — don't wait for a final transcript
- Begin agent reasoning on partial input when confidence is high
- Start TTS playback on the first sentence, not the full response
- Measure end-to-end time from last word spoken to first audio played

STT Client Setup: Configuration & Best Practices

Optimizing your Speech-to-Text client is crucial for a responsive and accurate voice agent. It involves careful configuration of connection settings, model parameters, and voice activity detection.

Connection & Transport

Establishing a robust connection and defining audio parameters are foundational for real-time STT.

- ### Connection Type

  WebSocket is preferred over HTTP streaming for real-time STT due to its lower latency and bidirectional communication, enabling continuous audio flow.

- ### Audio Format

  The most common format is **PCM 16-bit, 16kHz mono**. For telephony applications, some providers support **8kHz**.

- ### Audio Chunk Size

  Audio is typically sent in small chunks, usually **100–200ms** per send, to maintain low latency and allow the STT model to process continuously.

⚠ Voice Activity Detection (VAD)

Endpointing sensitivity dictates how long silence triggers the end-of-utterance. Too **aggressive** (e.g., short silence threshold) can cut off slow speakers, while too **lenient** (e.g., long silence threshold) can lead to noticeable pauses before the agent responds. Many providers allow configuring this threshold (e.g., `endpointing_ms` from 500ms to 1500ms).

Key Model Parameters

`language` / `locale`	Set explicitly (e.g., `en-US`); do not rely on auto-detect in production.
`model_tier`	Choose between **fast/base** (low latency, moderate accuracy) or **accurate/enhanced** (higher accuracy, slightly higher latency).
`punctuation_and_formatting`	Enable this for cleaner LLM input, making it easier for the agent to parse responses.
`profanity_filter`	Generally disable for agent use cases, as the agent should process all user input without censoring.
`word_timestamps`	Enable if you need word-level timings, useful for debugging or advanced UI features like highlighting.

ⓘ Partial vs. Final Transcripts

Partial transcripts are emitted continuously and may change; use them for early agent planning, not for executing actions. **Final transcripts** are stable and emitted after endpointing; these should be used as the definitive input for agent actions. Some providers use an `is_final` flag, others send distinct event types.

TTS Client Setup: Configuration, Parameters, and Best Practices

Optimizing your Text-to-Speech (TTS) client is paramount for creating fluid, natural, and responsive voice agent experiences. This involves careful consideration of connection types, model parameters, chunking strategies, and audio playback management.

Connection & Streaming

- **Streaming vs. Batch TTS:** Always prioritize **streaming** for voice agents. Batch processing introduces significant latency by waiting for the full response, whereas streaming allows for immediate audio synthesis and playback as text becomes available.
- **Transport: WebSocket** or **HTTP chunked transfer** are standard for streaming audio, enabling a continuous, low-latency data flow.
- **Audio Format:** `Opus` is often preferred for web applications due to its efficient compression and high quality at low bitrates. `PCM 16-bit` is common, especially for telephony, while `MP3` offers broader compatibility.
- **Sample Rate:** `24kHz` is standard for most neural TTS models, providing high fidelity. For telephony, `8kHz` is typically used to match network constraints.

Chunking Strategy

- **Split on Boundaries:** Avoid sending the full LLM output to TTS. Instead, **split the text on sentence boundaries** (or similar semantic units) to ensure natural prosody and responsive playback.
- **Minimum Chunk Size:** Aim for a minimum chunk size of around **one sentence or 10–15 tokens** to prevent audio from sounding choppy. Very small chunks can disrupt the natural flow of speech.
- **Flush on Punctuation:** Natural split points for flushing text to the TTS engine include periods (.), question marks (?), and exclamation points (!).
- **Input Streaming:** Some TTS providers support **input streaming**, allowing you to send tokens to the model as they arrive from the LLM, while others require complete sentences or full text for processing.

Key Model Parameters

`voice_id` / `voice_name`	Select a voice that aligns with your use case (e.g., warm, professional, neutral, regional accent) to convey the desired persona.
`model_tier`	Choose between standard, neural, or HD models. Neural models are generally the baseline for production-quality agents, while HD offers premium clarity and naturalness for enhanced experiences.
`speed` / `rate`	A natural speaking range is typically **0.9x to 1.1x**. Values outside this range can make the voice sound rushed or unnaturally slow and robotic.
`stability` / `similarity`	(Provider-specific) These parameters control the consistency versus expressiveness of the generated voice. Adjust them to achieve the desired emotional range and vocal flow.
Emotion / Style Tags	Some providers support `SSML` (Speech Synthesis Markup Language) or style prompts to control the tone, emotion, or specific speaking styles of the output.

Playback & Buffering

- **Initial Buffer:** Buffer **1–2 audio chunks** before starting playback to create a seamless experience and prevent underruns or stuttering.
- **Track Playback:** Actively track the current playback position. This is essential to support **barge-in** functionality, where new user input immediately cancels any queued audio and allows the agent to respond.
- **Audio Queue Management:** Implement robust handling for the audio queue, ensuring that new chunks arriving from the TTS engine are seamlessly added for playback while previous chunks are still being spoken.

By carefully configuring these aspects, you can significantly enhance the responsiveness and naturalness of your voice agent, delivering a truly conversational experience to users.

Barge-In: Letting Users Interrupt Naturally

What is barge-in?

Barge-in is when a user **starts speaking while the agent is still talking**, expecting it to stop and listen immediately. A voice agent that cannot handle this feels rigid and frustrating — especially when it gives long answers.

⊗ Without barge-in support, users are forced to wait through the entire agent response before they can correct, redirect, or ask a follow-up.

What good barge-in behavior looks like

01

Detect speech onset quickly

Voice activity detection (VAD) identifies that the user has started speaking mid-playback.

02

Stop TTS and cancel queued audio

Playback halts within a small time window; any buffered audio is discarded immediately.

03

Switch back to listening state

STT resumes on the new utterance with the correct conversational context — old and new turns are not mixed.

How a Voice Agent Should Phrase Its Responses

Voice replies need a fundamentally different style from text. Users are **listening**, not scanning — and they cannot scroll back.

Shorter and More Focused

Give one or two core points, then ask if the user wants more detail. Avoid dense paragraphs that work on screen but overwhelm in audio.

Explicit Confirmations

For critical details, confirm out loud: *"You asked about order 12345 to Mumbai, correct?"* — rather than assuming STT captured everything perfectly.

Leave Room for Corrections

Design prompts so the agent avoids long monologues. Short turns invite the user to correct, redirect, or confirm before the agent goes further.

The goal is a **conversational rhythm**, not a spoken document. Pacing for listening means shorter sentences, natural pauses, and frequent check-ins with the user.

STT Errors and Noisy Environments

Most common STT failure modes

Mis-heard Critical Fields

Names, locations, and numbers are especially vulnerable — particularly in noisy environments or with strong accents.

Shifting Partial Transcripts

Partial transcripts change as the model refines them. If the agent acts on early partials carelessly, it can be misled by text that later changes.

How the agent and UX should respond

- For high-stakes fields — IDs, dates, amounts — use **short confirmation turns** instead of acting immediately on what was heard
- Show **live transcripts** so users can see what was understood in real time
- Allow users to correct or repeat naturally, without forcing them to restart the interaction
- Treat partial transcripts as hints for planning, not as final inputs for action

Live transcripts serve double duty: they build user trust *and* give developers a debugging surface to identify where STT is struggling.

Voice in a Multi-Agent and Tool-Calling System

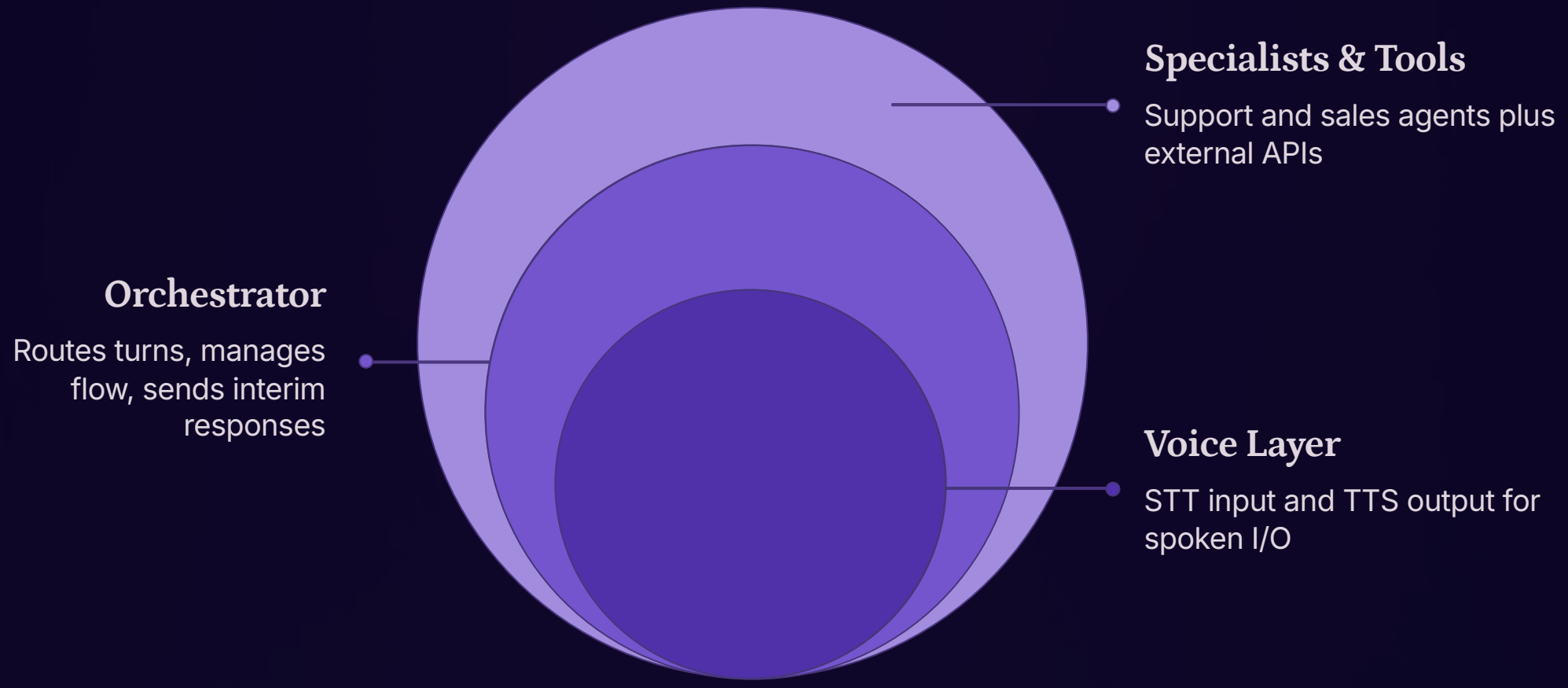
When a voice request triggers tools and multiple specialist agents, the orchestration layer does not need to be rebuilt — but it does need to be adapted for the spoken medium.

What stays the same

- The **orchestration logic** — which agents are called, which tools are used — remains unchanged. Voice is an additional input/output layer, not a new architecture.
- Support, sales, and other specialists still own their domains. The voice layer simply delivers transcripts in and plays audio out.

What needs to be adapted

- The Orchestrator may need to send **shorter, intermediate responses** while tools or other agents are still working — to avoid long silent gaps that feel like failures.
- Tool results may need to be **summarised differently for speech** — focusing on key fields and avoiding dense lists that are hard to follow when heard rather than read.



UI and Feedback Patterns for Voice

Even voice-first experiences need visual feedback

Clear state indicators and live transcripts are not optional extras — they are essential for users to know whether to talk, wait, or correct.

Listening

Waveform or indicator shows the system is actively capturing audio

Thinking

Spinner or pulse shows the agent is processing — don't leave users in silence

Speaking

Animated indicator shows TTS is playing; barge-in is now available

Transcripts and captions also provide a **bridge to text-based debugging**: what you see in traces and what users experience in voice stay aligned.

Where to Start: Adding Voice to an Existing Agent

Given all these moving parts, a phased approach dramatically reduces risk and accelerates learning.

Pick Short, High-Value Tasks First

Start with checking status, quick comparisons, or simple FAQs — where the cost of mis-hearing is low and conversation flows are compact. Avoid complex branching conversations until the basics are solid.

Tune the Core Voice Behaviors

Use these early flows to calibrate latency budgets, barge-in thresholds, speaking style, and confirmation patterns. Real user behavior will surface issues that design reviews miss.

Collect Real Metrics and Iterate

Early flows give you concrete data on latency, STT error rates, and user behavior. Use these to refine prompts, tools, and routing — and to identify which parts of your multi-agent system benefit most from voice versus remaining text-only or UI-driven.

The goal of early voice flows is not perfection — it is instrumentation. Every real interaction teaches you something that no amount of upfront design can predict.